

Time-to-First-Spike Identity Mapping

刘沛雨

2025.6.8

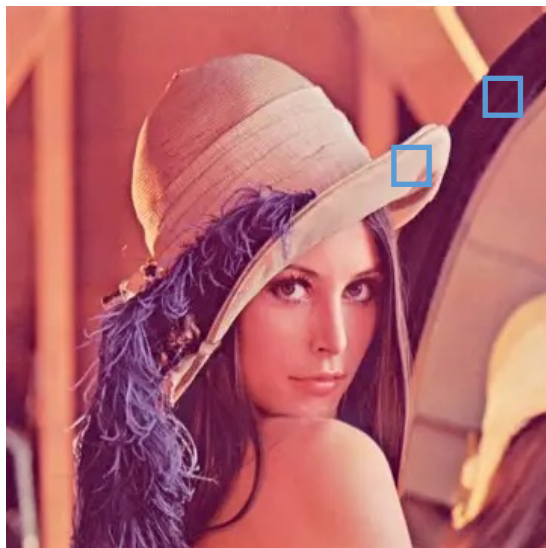
Outline

- Background
 - ✓ Rate coding & temporal coding
 - ✓ SNN Training
- Methods
 - ✓ Time-to-first-spike (TTFS) coding
 - ✓ Two-stage integrate-and-fire (IF) neuron
 - ✓ Identity mapping between ReLU and TTFS Neuron
 - ✓ Gradient flow analysis
- Results
 - ✓ Reproduction
- Discussion

How spikes encode information?

Spike itself does not contain any information

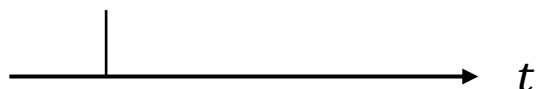
- Every spike looks similar in shape
- **Frequency** and **timing** are important
 - ✓ Through frequency – *rate coding*
 - ✓ Through timing – *temporal coding*



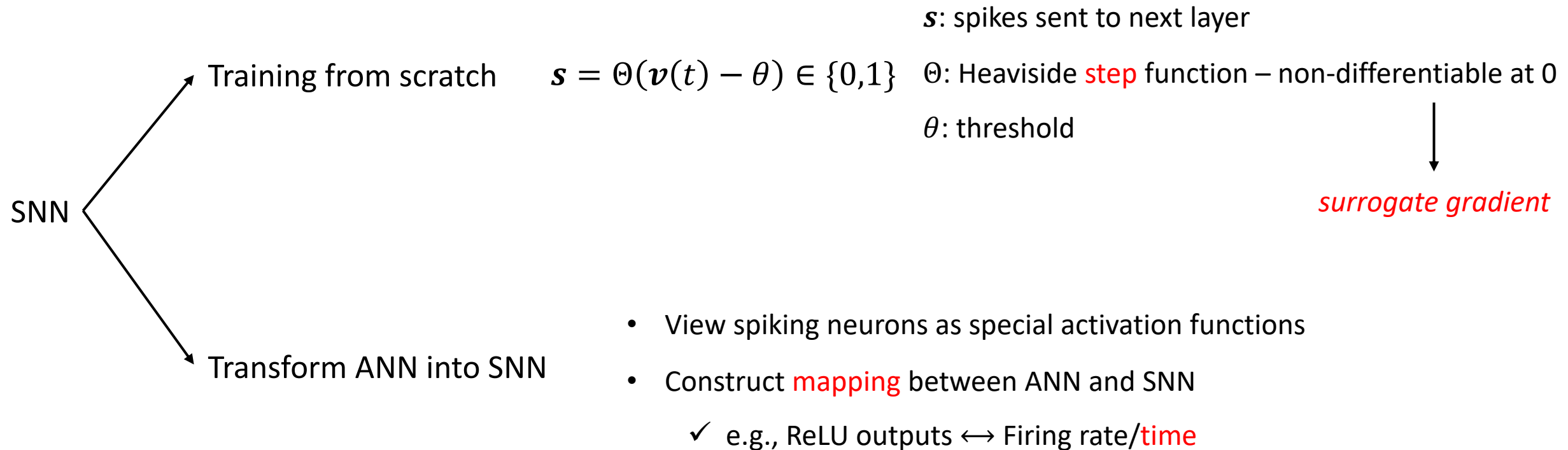
rate coding



temporal coding



SNN Training



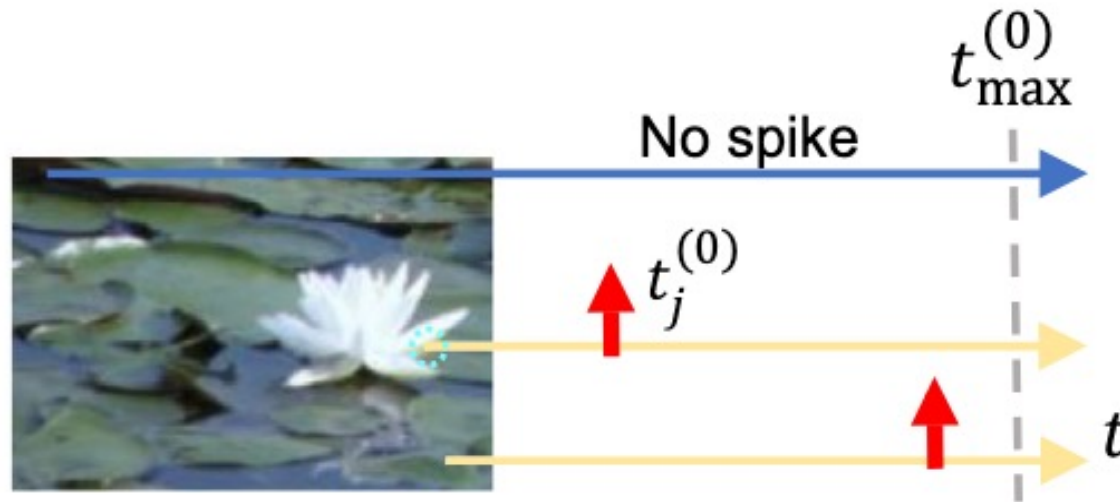
High-performance deep spiking neural networks with 0.3 spikes per neuron

Received: 20 November 2023

Accepted: 29 July 2024

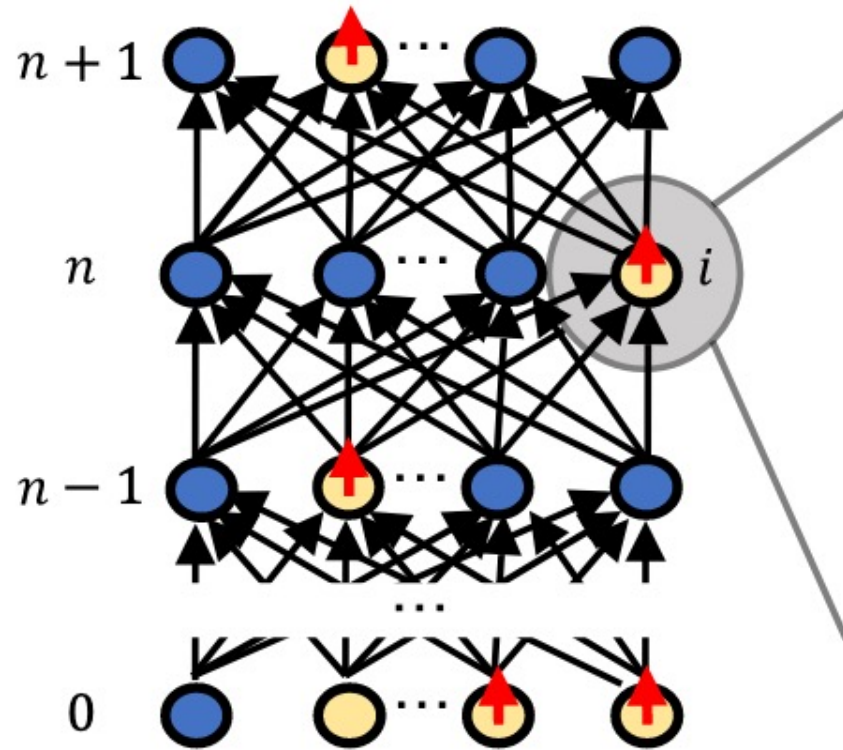
Ana Stanojevic^{1,2}, Stanisław Woźniak ¹✉, Guillaume Bellec ^{2,3},
Giovanni Cherubini ¹, Angeliki Pantazi ¹ & Wulfram Gerstner^{2,3}

Time-to-First-Spike (TTFS) Coding

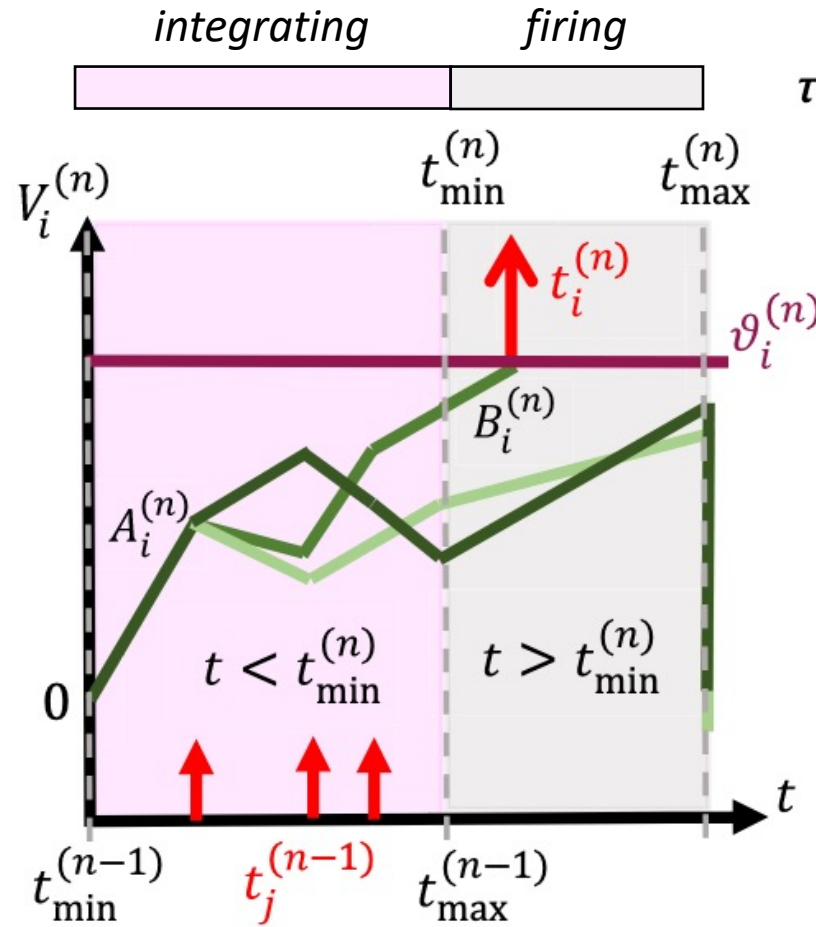


High intensity leads to early firing.

Two-stage IF Neuron



Feed-Forward Network

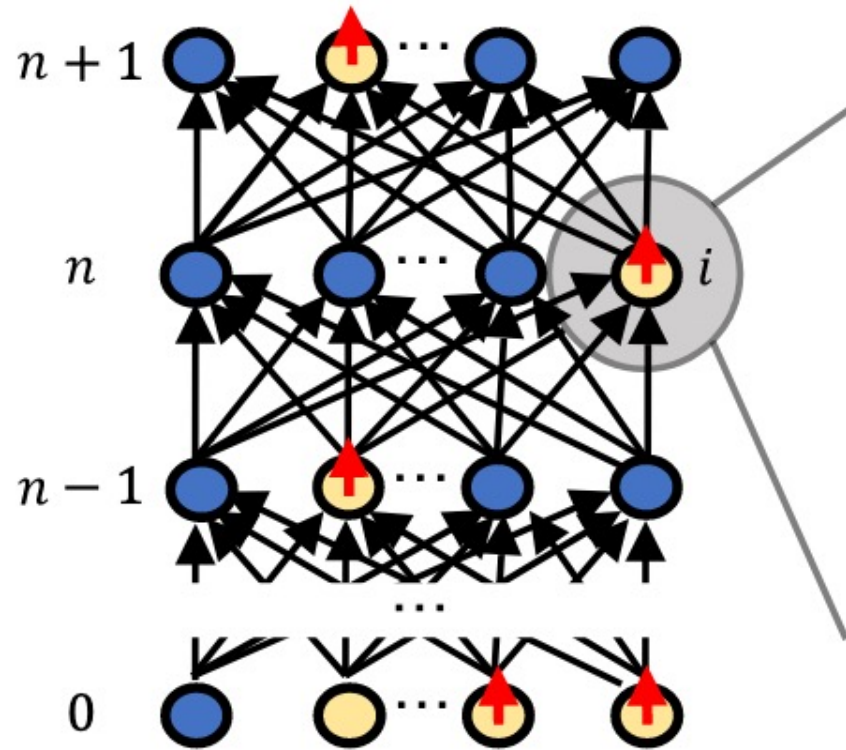


Neuronal Dynamics

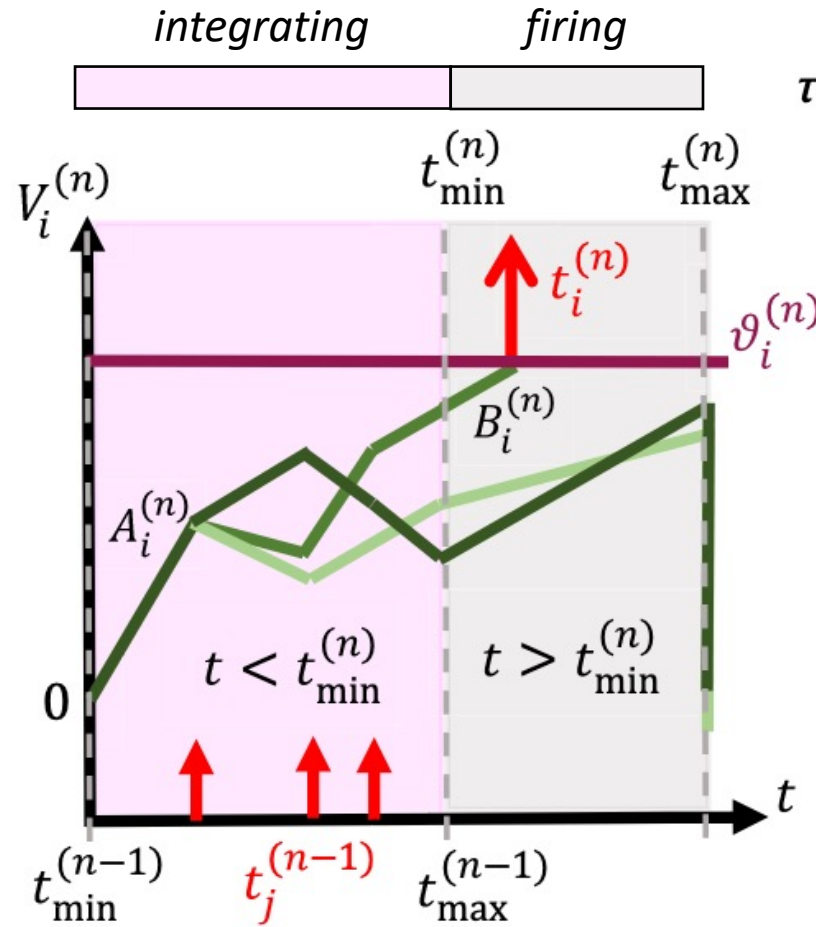
$$\tau_c \frac{dV_i^{(n)}}{dt} = \begin{cases} A_i^{(n)} + \sum_j W_{ij}^{(n)} H(t - t_j^{(n-1)}) \\ B_i^{(n)} \end{cases}$$

- Assume $\tau_c = 1$
- Initial slope: $A_i^{(n)}$
- After entering *firing* phase, potential grows constantly with $B_i^{(n)}$
- Each neuron fires **at most** once

Two-stage IF Neuron



Feed-Forward Network



Neuronal Dynamics

$$\tau_c \frac{dV_i^{(n)}}{dt} = \begin{cases} A_i^{(n)} + \sum_j W_{ij}^{(n)} H(t - t_j^{(n-1)}) \\ B_i^{(n)} \end{cases}$$

- Constraint #1: $t_{\min}^{(n)} = t_{\max}^{(n-1)}$
- Constraint #2: no neurons fire **before** $t_{\min}^{(n)}$ (through dynamic threshold) and **after** $t_{\max}^{(n)}$

$$\vartheta_i^{(n)} = \text{def } \widetilde{\vartheta}_i^{(n)} - D_i^{(n)}$$

Identity Mapping Between ReLU and TTFS Neuron

$$\tau_c \frac{dV_i^{(n)}}{dt} = \begin{cases} A_i^{(n)} + \sum_j W_{ij}^{(n)} H(t - t_j^{(n-1)}) \\ B_i^{(n)} \end{cases}$$

$$\vartheta_i^{(n)} = \text{def } \tilde{\vartheta}_i^{(n)} - D_i^{(n)}$$

$$A_i^{(n)} = 0$$

$\Leftrightarrow$

$$\begin{array}{c} x_j^{(n)} \text{ (after ReLU)} \\ \frac{t_{\max}^{(n)} - t_i^{(n)}}{\tau_c} \end{array} = \sum_j \begin{array}{c} w_{ij}^{(n)} \\ \frac{W_{ij}^{(n)}}{B_i^{(n)}} \end{array} \cdot \begin{array}{c} x_j^{(n-1)} \\ \frac{t_{\max}^{(n-1)} - t_j^{(n-1)}}{\tau_c} \end{array} + \begin{array}{c} bias_i^{(n)} \\ \frac{t_{\max}^{(n-1)} - t_{\min}^{(n-1)}}{\tau_c} - \frac{\vartheta_i^{(n)}}{B_i^{(n)}} \end{array}$$

$t_i^{(n)}$: time when neuron i in layer n fires (assume it fires).

- Constraint #2: no neurons fire **before** $t_{\min}^{(n)}$ (through dynamic threshold) and **after** $t_{\max}^{(n)}$

A two-stage IF neuron fires in SNN $\Leftrightarrow$ Corresponding ReLU neuron produces a positive output in ANN

Main Findings - Gradients are still not equivalent.

- Pervious research used similar neuron models and methods to derive identity mapping
 - ✓ But only achieved equivalence **at initialization**
 - ✓ Problems must lie in backpropagation

$$x_j^{(n)} \text{ (after ReLU)} = \sum_j \frac{w_{ij}^{(n)}}{B_i^{(n)}} \cdot \frac{t_{\max}^{(n-1)} - t_j^{(n-1)}}{\tau_c} + \frac{t_{\max}^{(n-1)} - t_{\min}^{(n-1)}}{\tau_c} - \frac{\vartheta_i^{(n)}}{B_i^{(n)}}$$

$$\frac{d\mathcal{L}}{dW^{(n)}} = \frac{d\mathcal{L}}{dV^{(N+1)}} \frac{dV^{(N+1)}}{dt^{(N)}} \frac{dt^{(N)}}{dt^{(N-1)}} \cdots \frac{dt^{(n+1)}}{dt^{(n)}} \frac{dt^{(n)}}{dW^{(n)}},$$

$$\frac{dt^{(n)}}{dt^{(n-1)}} = M^{(n-1)} \cdot \frac{1}{B^{(n)}} \cdot W^{(n)} = M^{(n-1)} w^{(n)}.$$

$$M^{(n-1)} = \text{diag}\{m_1^{(n)}, m_2^{(n)}, \dots, m_N^{(n)}\}, m_i^{(n)} \in \{0,1\} \text{ - gradient gating}$$

$$B^{(n)} = \text{diag}\{B_1^{(n)}, B_2^{(n)}, \dots, B_N^{(n)}\}, m_i \in \{0,1\}$$

$w^{(n)}$: weights in ANN corresponding to $W^{(n)}$ in SNN

Main Findings - Gradients are equivalent only when $B^{(n)} = 1$.

- Previous research set **different** $B_i^{(n)}$ for each neuron i in layer n
 - ✓ leading to “unequal” gradient flow
 - ✓ while for ReLU the gradient is always **1** under the same condition (firing $\Leftrightarrow$ producing positive outputs)

$$\frac{d\mathcal{L}}{dW^{(n)}} = \frac{d\mathcal{L}}{dV^{(N+1)}} \frac{dV^{(N+1)}}{dt^{(N)}} \frac{dt^{(N)}}{dt^{(N-1)}} \cdots \frac{dt^{(n+1)}}{dt^{(n)}} \frac{dt^{(n)}}{dW^{(n)}},$$

$$M^{(n-1)} = \text{diag}\{m_1^{(n)}, m_2^{(n)}, \dots, m_N^{(n)}\}, m_i^{(n)} \in \{0,1\} - \text{gradient gating}$$

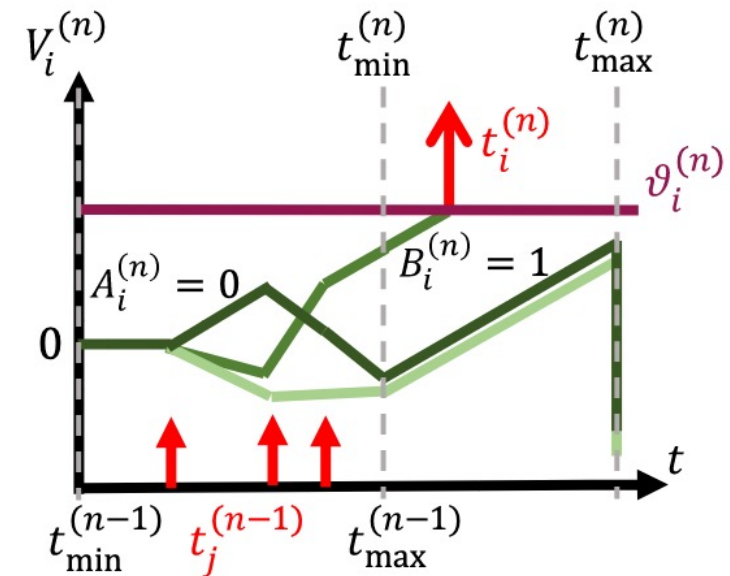
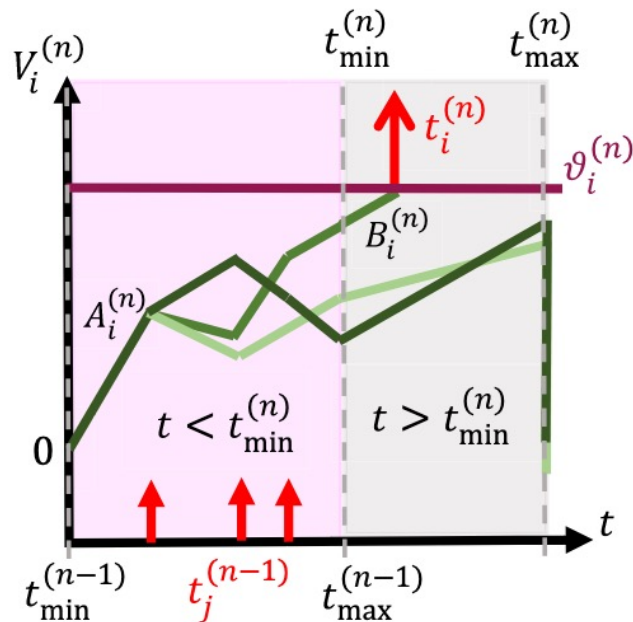
$$B^{(n)} = \text{diag}\{B_1^{(n)}, B_2^{(n)}, \dots, B_N^{(n)}\}, m_i \in \{0,1\}$$

$$\frac{dt^{(n)}}{dt^{(n-1)}} = M^{(n-1)} \cdot \frac{1}{B^{(n)}} \cdot W^{(n)} = M^{(n-1)} w^{(n)}.$$

$w^{(n)}$: weights in ANN corresponding to $W^{(n)}$ in SNN

Main Findings - Gradients are equivalent only when $B^{(n)} = 1$.

- Pervious research set different $B_i^{(n)}$ for each neuron i in layer n
 - ✓ leading to “unequal” gradient flow
 - ✓ while for ReLU the gradient is always **1** under the same condition (firing $\Leftrightarrow$ producing positive outputs)
 - ✓ thus setting $B^{(n)}$ to 1



Performance on MNIST

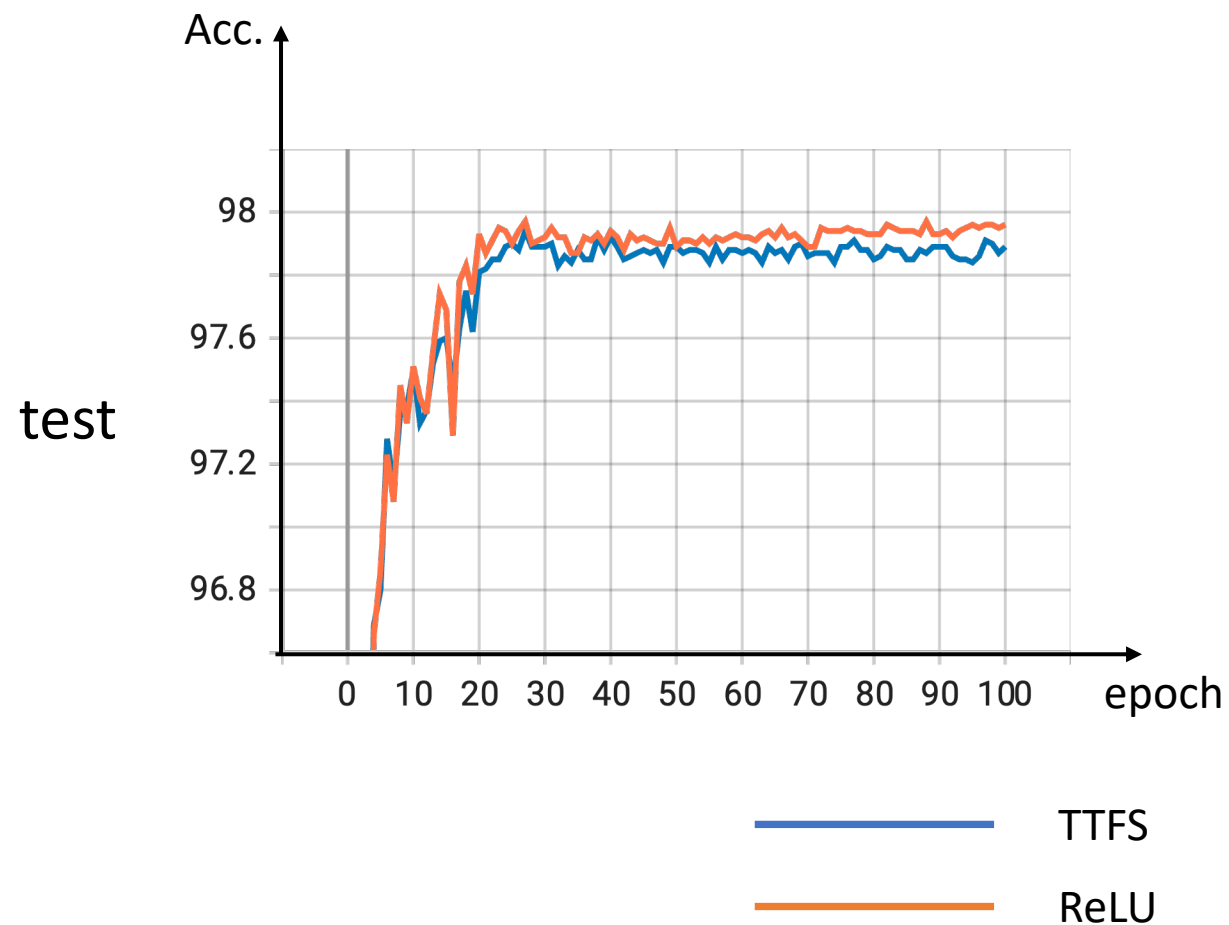
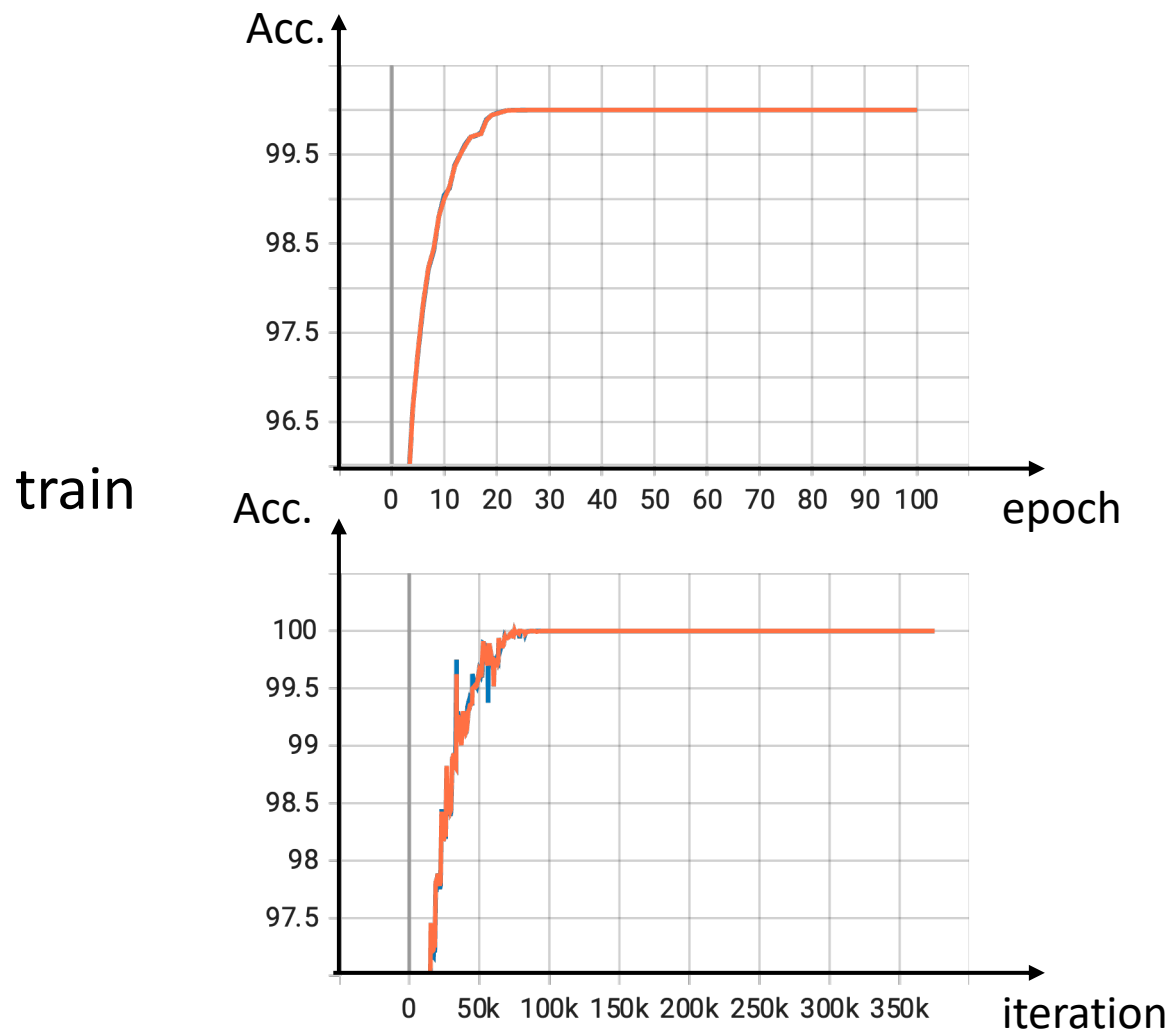
Model	Arch.	Test acc. [%] (from paper)	Test acc. [%] (reproduced)
ReLU	FC2	98.30	98.38
TTFS	FC2	98.30	98.40
ReLU	FC16	98.43 $\pm$ 0.07	98.14
TTFS	FC16	98.43 $\pm$ 0.07	98.16
ReLU	VGG16	99.57 $\pm$ 0.01	99.59
TTFS	VGG16	99.58 $\pm$ 0.01	99.50

Performance on FashionMNIST

Model	Arch.	Test acc. [%] (from paper)	Test acc. [%] (reproduced)
ReLU	FC2	90.14	90.22
TTFS	FC2	90.14	90.31
ReLU	LeNet5	90.91 ± 0.17	90.73
TTFS	LeNet5	90.94 ± 0.25	90.80

Implementation is based on PyTorch; Codes will be released on [GitHub](#)

Training Equivalence (demo. on a 8-layer MLP)



Discussion

Summary

- Potential in the *firing* phase scales the gradients during backpropagation
- Always make sure that these potential has same gradient with that of ANN activation functions

Future work

- Reproduction
 - ✓ Visualization
 - ✓ Testing on larger benchmarks (e.g., CIFAR10/100, ImageNet, PLACE365, etc.)
 - ✓ Robustness tests
 - ✓ ANN2SNN conversion in TTFS
- Other possible directions
 - ✓ Implementing TTFS on ResNet
 - ✓ Better Compatibility with BatchNorm

Reference

- Stanojevic A. *et al.* High-performance deep spiking neural networks with 0.3 spikes per neuron. *Nat Commun.* 2024 Aug 9;15(1):6793.
- Stanojevic A. *et al.* An exact mapping from ReLU networks to spiking neural networks. *Neural Netw.* 2023 Nov;168:74-88.

Thank you!

THANK
YOU

Q&A